

SRI LANKA INSTITUTE OF INFORMATION TECHNOLOGY

Faculty of Computing – Department of Computer Science & Software Engineering

SE3082 – Parallel Computing | Lab Report 9

Student Name / ID:	IT24610803	Academic Year / Sem:	Year 3, Semester 2
Module:	SE3082 – Parallel Computing	Lab Topic:	Introduction to CUDA (Google Colab Edition)
Environment:	Google Colab – NVIDIA Tesla T4 GPU (~15 GB VRAM, Compute Capability 7.5)		

Exercise 1: Environment Setup & Smoke Test (hello.cu)

I set up the Google Colab environment by selecting the **T4 GPU** hardware accelerator. I verified the GPU availability using `!nvidia-smi` and the compiler with `!nvcc --version`. Then, I wrote a simple smoke test kernel that launches 1 block containing 4 threads (`hello<<<1, 4>>>`) to test printing directly from the GPU.

Program Output:

```
Hello from GPU thread 0
Hello from GPU thread 1
Hello from GPU thread 2
Hello from GPU thread 3
```

Observation: Each thread runs concurrently and prints its thread ID using `threadIdx.x`. The outputs can appear in any order because thread execution is asynchronous on the GPU.

Exercise 2: Simple Calculation Program (ex2_add.cu)

In this exercise, I implemented the basic host-device memory workflow to add two integers ($a=2$ and $b=7$) on the GPU using a single thread (`add<<<1, 1>>>`).

- Allocated device memory with `cudaMalloc` for `d_a`, `d_b`, and `d_c`.
- Copied values from host to device using `cudaMemcpy(..., cudaMemcpyHostToDevice)`.
- Executed the kernel and checked for errors with `cudaGetLastError()`.
- Copied the result back to host memory using `cudaMemcpy(..., cudaMemcpyDeviceToHost)`.

```
Result is 9
```

Answers to Think Questions:

1. Why do we need separate variables `d_a`, `d_b`, `d_c` on the device?

The CPU (host) and GPU (device) have separate physical memories (System RAM vs GPU VRAM). Pointers to CPU memory addresses cannot be accessed directly by GPU threads. Therefore, we must allocate space in GPU memory using `cudaMalloc` and pass device pointers to the kernel.

2. Why pass `&a` to `cudaMemcpy` but `d_a` to kernel launch?

a is a local variable stored on the CPU stack, so &a (its memory address, type `int*`) is passed as the source pointer. On the other hand, d_a is already a pointer variable (type `int*`) pointing to GPU memory, so it is passed directly by value into the kernel.

3. How many threads run the kernel, and what if we use 10 threads?

Currently, exactly 1 thread runs the kernel (`<<<1, 1>>>`). Changing to `<<<1, 10>>>` would run 10 threads.

The answer would still be 9 because all 10 threads compute $2 + 7 = 9$ and write to the same memory location, but it results in redundant writes.

Exercise 3: Vector Addition with 512 Blocks and 1 Thread (ex3_blocks.cu)

In this program, two vectors of 512 elements are added. The kernel is launched with **512 blocks of 1 thread each** (`add<<<512, 1>>>`). Inside the kernel, each thread uses `blockIdx.x` to determine which element to compute.

```
83 + 77 = 160
86 + 34 = 120
77 + 90 = 167
...
Verification: 0 mismatches out of 512
```

Answers to Predict Before You Run:

- 1. Total blocks and threads:** The launch configuration `add<<<N, 1>>>` launches **512 blocks** with 1 thread each, totaling **512 threads**.
- 2. Warp execution impact:** NVIDIA GPUs execute threads in groups of 32 called warps. When a block has only 1 thread, an entire warp of 32 threads is scheduled on the Streaming Multiprocessor, but **31 threads remain completely idle**. This results in very poor hardware utilization (only $1/32 \approx 3.125\%$ warp efficiency) and extra overhead for scheduling 512 blocks.

Exercise 4: Vector Addition with 1 Block and 512 Threads (ex4_threads.cu)

Here, the same addition is performed using **1 block of 512 threads** (`addT<<<1, 512>>>`). Each thread finds its element using `threadIdx.x`.

```
0) 83 + 77 = 160
...
511) 67 + 6 = 73
Verification: 0 mismatches out of 512
```

Experiment – Finding the GPU Block Limit:

When I tested changing `#define N 512` to `#define N 2048` in a single block (`addT<<<1, 2048>>>`), the program printed:

```
Kernel launch failed: invalid argument
```

Experiment Analysis:

The error check right after the kernel launch (`cudaGetLastError()`) caught the failure. Modern NVIDIA GPUs have a hardware architectural limit of **1024 threads per block** (`maxThreadsPerBlock = 1024`). Requesting 2048 threads in a single block exceeds this hardware limit, causing the CUDA runtime driver to reject the launch configuration.

Exercise 5: Multiplying Two 10,000,000-Element Vectors (ex5_vecmul.cu)

To process 10,000,000 elements, I combined both blocks and threads:

- **Threads per Block:** 512 threads
- **Global Thread Index:** `int i = blockIdx.x * blockDim.x + threadIdx.x;`
- **Blocks Needed (Rounded UP):** `int blocks = (N + 511) / 512;` $\implies \mathbf{19,532 \text{ ext{ blocks}}}$
- **Total Threads Launched:** $19,532 \times 512 = 10,000,384$ threads.
- **Boundary Check:** Since 10,000,384 threads were launched for 10,000,000 elements, the condition `if (i < n)` is strictly necessary to prevent the last 384 threads from accessing out-of-bounds memory.

Program Output:

```
Launching 19532 blocks x 512 threads
9999000) 76 x 52 = 3952
...
9999999) 31 x 73 = 2263
Verification: 0 mismatches out of 10000000
```

Answer to Think Question:

Why use multiples of 32 for threads per block?

GPUs execute threads in warps of 32. Using a block size that is a multiple of 32 (like 512) ensures all warps are fully utilized. If 500 threads per block were used instead of 512, the last warp in every block would only have 20 active threads while 12 threads remain idle ($500 \bmod 32 = 20$). Across 20,000 blocks, this would waste $20,000 \times 12 = 240,000$ warp execution slots.

Exercise 6: Multiplying Two 10,000 × 10,000 Matrices Element-by-Element (ex6_matrix.cu)

In this exercise, I multiplied two 2D matrices element by element (`C[r][c] = A[r][c] * B[r][c]`) containing 100,000,000 integers (400 MB per matrix).

- **Threads per Block:** `dim3 threadsPerBlock(32, 16);` ($32 \times 16 = 512$ threads/block).
- **Thread Coordinates:** Column: `col = blockIdx.x * blockDim.x + threadIdx.x;` Row: `row = blockIdx.y * blockDim.y + threadIdx.y;`
- **Flat Index Calculation:** `int idx = row * cols + col;`
- **Boundary Guard:** `if (row < rows && col < cols)`
- **Grid Dimensions:** Columns: $(10000 + 31) / 32 = \mathbf{313 \text{ ext{ blocks}}}$, Rows: $(10000 + 15) / 16 = \mathbf{625 \text{ ext{ blocks}}}$. Total: $313 \times 625 = \mathbf{195,625 \text{ ext{ blocks}}}$ (100,160,000 threads).

Program Output:

```
Grid: 313 x 625 blocks
C[9999][9000] 29 x 56 = 1624
...
C[9999][9999] 86 x 21 = 1806
Verification: 0 mismatches out of 100000000
```

Conclusion & Verification Summary

Exercise	Configuration	Elements Computed	Verification Result	Status
Exercise 1	1 block $\times$ 4 threads	4	Printed 4 thread IDs	PASSED
Exercise 2	1 block $\times$ 1 thread	1	Result is 9	PASSED
Exercise 3	512 blocks $\times$ 1 thread	512	0 mismatches out of 512	PASSED
Exercise 4	1 block $\times$ 512 threads	512	0 mismatches out of 512	PASSED
Exercise 5	19,532 blocks $\times$ 512 threads	10,000,000	0 mismatches out of 10,000,000	PASSED
Exercise 6	313 $\times$ 625 blocks of 32 $\times$ 16	100,000,000	0 mismatches out of 100,000,000	PASSED